

Sorting Algorithms Lab

Data Structures & Algorithms

Lab 10 -- Implementing Common Sorting Algorithms

Duration: 2 hours (2 challenges)

Lab Structure

Each challenge follows this format:

1. **Challenge Presentation** (5 min)
2. **Your Implementation Time** (25 min)
3. **Pseudocode Review** (5 min)
4. **Solution Walkthrough** (10 min)
5. **Code Explanation** (15 min)

Total per challenge: ~60 minutes

Today's Challenges

Challenge 1: Student Scores -- Bubble Sort and Selection Sort

- Sort an array of student exam scores using **Bubble Sort**
- Re-sort the same array using **Selection Sort**
- Count and compare the number of **swaps** each algorithm performs

Challenge 2: Card Hand -- Insertion Sort

- Sort a hand of playing cards (integers) using **Insertion Sort**
- Track the number of **shifts** performed
- Compare all three sorting algorithms on the same data

Quick Recap: Sorting Fundamentals

A **sorting algorithm** arranges elements of a list in a particular order (ascending/descending).

Key Properties

Property	Meaning
In-place	Uses $O(1)$ extra memory (sorts within the array)
Stable	Equal elements keep their original relative order
Adaptive	Runs faster on partially sorted input

Simple Sorts -- All $O(n^2)$ Worst Case

Algorithm	Best	Average	Worst	Stable?	Adaptive?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes	Yes

Challenge 1

Student Scores -- Bubble Sort and Selection Sort

Challenge 1: Problem Statement

Scenario: You have exam scores for 8 students that need to be ranked from lowest to highest.

```
Scores: {64, 34, 25, 12, 22, 11, 90, 45}
```

Tasks:

1. Implement **Bubble Sort** to sort the scores. Print the array after each pass. Count total swaps.
2. Reset the array to its original order.
3. Implement **Selection Sort** to sort the same scores. Print the array after each pass. Count total swaps.
4. Print a comparison of total swaps for each algorithm.

Time: 25 minutes

Challenge 1: Think About It First

Bubble Sort -- "Bubble up" the largest element each pass

Pass 1: Compare adjacent pairs, swap if out of order.
The largest element "bubbles" to the end.
Pass 2: Repeat for the remaining $n-1$ elements.
...continue until no swaps are needed.

Selection Sort -- "Select" the minimum each pass

Pass 1: Find the smallest element, swap it into position 0.
Pass 2: Find the smallest in remaining, swap into position 1.
...continue until all positions are filled.

Challenge 1: Bubble Sort Trace

Original: [64, 34, 25, 12, 22, 11, 90, 45]

Pass 1: Compare and swap adjacent pairs left to right

(64,34)→swap (64,25)→swap (64,12)→swap (64,22)→swap
(64,11)→swap (64,90)→no (90,45)→swap

Result: [34, 25, 12, 22, 11, 64, 45, 90] Swaps: 6

Pass 2: [25, 12, 22, 11, 34, 45, 64, 90] Swaps: 4

Pass 3: [12, 22, 11, 25, 34, 45, 64, 90] Swaps: 2

Pass 4: [12, 11, 22, 25, 34, 45, 64, 90] Swaps: 1

Pass 5: [11, 12, 22, 25, 34, 45, 64, 90] Swaps: 1

Pass 6: No swaps needed → STOP early.

Total Bubble Sort swaps: 14

Challenge 1: Selection Sort Trace

Original: [64, 34, 25, 12, 22, 11, 90, 45]

Pass 1: Find min in [0..7] → 11 at index 5. Swap arr[0] ↔ arr[5].
[11, 34, 25, 12, 22, 64, 90, 45] Swaps: 1

Pass 2: Find min in [1..7] → 12 at index 3. Swap arr[1] ↔ arr[3].
[11, 12, 25, 34, 22, 64, 90, 45] Swaps: 1

Pass 3: Find min in [2..7] → 22 at index 4. Swap arr[2] ↔ arr[4].
[11, 12, 22, 34, 25, 64, 90, 45] Swaps: 1

Pass 4: Find min in [3..7] → 25 at index 4. Swap arr[3] ↔ arr[4].
[11, 12, 22, 25, 34, 64, 90, 45] Swaps: 1

Pass 5: min is 34, already at index 4. No swap.

Pass 6: Find min in [5..7] → 45 at index 7. Swap arr[5] ↔ arr[7].
[11, 12, 22, 25, 34, 45, 90, 64] Swaps: 1

Pass 7: Find min in [6..7] → 64 at index 7. Swap arr[6] ↔ arr[7].
[11, 12, 22, 25, 34, 45, 64, 90] Swaps: 1

Total Selection Sort swaps: 6

Challenge 1: Pseudocode

```
ALGORITHM BubbleSort(arr, n)
    swaps = 0
    For i = 0 to n-2:
        swapped = false
        For j = 0 to n-2-i:
            If arr[j] > arr[j+1]:
                Swap arr[j] and arr[j+1]
                swaps++, swapped = true
        Print array after this pass
        If not swapped: BREAK    // Early termination
    Return swaps
```

```
ALGORITHM SelectionSort(arr, n)
    swaps = 0
    For i = 0 to n-2:
        minIdx = i
        For j = i+1 to n-1:
            If arr[j] < arr[minIdx]: minIdx = j
        If minIdx != i:
            Swap arr[i] and arr[minIdx]
            swaps++
        Print array after this pass
    Return swaps
```

Challenge 1: Solution Code -- Helpers

```
#include <stdio.h>

#define SIZE 8

void printArray(int arr[], int n) {
    printf("  ");
    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i < n - 1) printf(", ");
    }
    printf("]\n");
}

void copyArray(int src[], int dest[], int n) {
    for (int i = 0; i < n; i++) dest[i] = src[i];
}

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

Challenge 1: Solution Code -- Bubble Sort

```
int bubbleSort(int arr[], int n) {
    int swaps = 0;

    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;

        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(&arr[j], &arr[j + 1]);
                swaps++;
                swapped = 1;
            }
        }

        printf("  Pass %d:", i + 1);
        printArray(arr, n);

        if (!swapped) {
            printf("  No swaps in pass %d -- array is sorted!\n", i + 1);
            break;
        }
    }
    return swaps;
}
```

Challenge 1: Solution Code -- Selection Sort

```
int selectionSort(int arr[], int n) {  
    int swaps = 0;  
  
    for (int i = 0; i < n - 1; i++) {  
        int minIdx = i;  
  
        for (int j = i + 1; j < n; j++) {  
            if (arr[j] < arr[minIdx])  
                minIdx = j;  
        }  
  
        if (minIdx != i) {  
            swap(&arr[i], &arr[minIdx]);  
            swaps++;  
        }  
  
        printf(" Pass %d:", i + 1);  
        printArray(arr, n);  
    }  
    return swaps;  
}
```

Challenge 1: Solution Code -- Main

```
int main() {
    int original[] = {64, 34, 25, 12, 22, 11, 90, 45};
    int arr[SIZE];
    int swapsBubble, swapsSelection;

    printf("=== Challenge 1: Bubble Sort vs Selection Sort ===\n\n");
    printf("Original array:");
    printArray(original, SIZE);

    // Bubble Sort
    printf("\n--- Bubble Sort ---\n");
    copyArray(original, arr, SIZE);
    swapsBubble = bubbleSort(arr, SIZE);
    printf("    Total swaps: %d\n", swapsBubble);

    // Selection Sort
    printf("\n--- Selection Sort ---\n");
    copyArray(original, arr, SIZE);
    swapsSelection = selectionSort(arr, SIZE);
    printf("    Total swaps: %d\n", swapsSelection);

    printf("\n--- Comparison ---\n");
    printf("    Bubble Sort swaps:    %d\n", swapsBubble);
    printf("    Selection Sort swaps: %d\n", swapsSelection);

    return 0;
}
```

Challenge 1: Code Explanation

Part 1: Why Bubble Sort Uses More Swaps

Bubble Sort swaps **every** adjacent out-of-order pair it encounters. An element far from its correct position gets swapped many times -- once per position it needs to move.

```
64 needs to move from index 0 to index 6.  
Pass 1 alone swaps 64 five times: positions 0→1→2→3→4→5.
```

Selection Sort makes **at most one swap per pass** -- it finds the minimum first, then places it directly. It never swaps an element to an intermediate position.

Swap count: Selection Sort always does $\leq n-1$ swaps. Bubble Sort can do up to $O(n^2)$ swaps.

Challenge 1: Code Explanation

Part 2: Early Termination in Bubble Sort

```
if (!swapped) break;
```

If a full pass produces zero swaps, the array is already sorted. This makes Bubble Sort **adaptive** -- it runs in $O(n)$ on an already-sorted array.

Selection Sort has no such optimization. Even if the array is fully sorted, it still scans for the minimum in each pass -- always $O(n^2)$ comparisons.

Part 3: Stability

Bubble Sort is stable: Equal elements are never swapped (we only swap when `arr[j] > arr[j+1]`, not `>=`).

Selection Sort is unstable: The long-range swap can jump over equal elements. Example: `[4a, 4b, 1]` → after pass 1, `1` swaps with `4a` → `[1, 4b, 4a]` -- the two 4s reversed order.

Challenge 2

Card Hand -- Insertion Sort

Challenge 2: Problem Statement

Scenario: You are picking up playing cards one by one and inserting each card into its correct position in your hand (like you would in real life).

Cards picked up (in order): {7, 3, 9, 1, 5, 8, 2, 6}

Tasks:

1. Implement **Insertion Sort** to sort the cards
2. Print the array after each insertion, showing which card was inserted and how many shifts it required
3. Count total comparisons and total shifts
4. After sorting, run all three algorithms (Bubble, Selection, Insertion) on the same data and compare swap/shift counts

Time: 25 minutes

Challenge 2: Think About It First

Insertion Sort builds a **sorted region** from left to right. Each new element is "inserted" into its correct position within the sorted region by shifting larger elements to the right.

Start: [7] 3 9 1 5 8 2 6 Sorted region: just 7.

Insert 3: $3 < 7$, shift 7 right. [3, 7] 9 1 5 8 2 6

Insert 9: $9 > 7$, no shift needed. [3, 7, 9] 1 5 8 2 6

Insert 1: $1 < 9, 7, 3$. Shift all 3. [1, 3, 7, 9] 5 8 2 6

Insert 5: $5 < 9, 7$. Shift 2. [1, 3, 5, 7, 9] 8 2 6

Insert 8: $8 < 9$. Shift 1. [1, 3, 5, 7, 8, 9] 2 6

Insert 2: $2 < 9, 8, 7, 5, 3$. Shift 5. [1, 2, 3, 5, 7, 8, 9] 6

Insert 6: $6 < 9, 8, 7$. Shift 3. [1, 2, 3, 5, 6, 7, 8, 9]

Total shifts: $1 + 0 + 3 + 2 + 1 + 5 + 3 = 15$

Challenge 2: Pseudocode

```
ALGORITHM InsertionSort(arr, n)
    totalShifts = 0, totalComparisons = 0

    For i = 1 to n-1:
        key = arr[i]
        j = i - 1
        shifts = 0

        While j >= 0 AND arr[j] > key:
            arr[j + 1] = arr[j]    // Shift right
            j--
            shifts++
            totalComparisons++

        If j >= 0:                // One extra comparison that failed
            totalComparisons++

        arr[j + 1] = key          // Place key in correct position
        totalShifts += shifts
        Print: "Inserted key, shifts done, current array"

    Print totalComparisons, totalShifts

END ALGORITHM
```

Challenge 2: Solution Code -- Insertion Sort

```
int insertionSort(int arr[], int n) {
    int totalShifts = 0;

    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        int shifts = 0;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j]; // Shift element right
            j--;
            shifts++;
        }
        arr[j + 1] = key; // Place key in correct position
        totalShifts += shifts;

        printf("  Insert %d (%d shift%s):", key, shifts,
            shifts == 1 ? "" : "s");
        printArray(arr, n);
    }
    return totalShifts;
}
```

Challenge 2: Solution Code -- Main

```
int main() {
    int original[] = {7, 3, 9, 1, 5, 8, 2, 6};
    int arr[SIZE];
    int swapsBubble, swapsSelection, shiftsInsertion;

    printf("=== Challenge 2: Insertion Sort ===\n\n");
    printf("Original array:");
    printArray(original, SIZE);

    printf("\n--- Insertion Sort ---\n");
    copyArray(original, arr, SIZE);
    shiftsInsertion = insertionSort(arr, SIZE);
    printf("  Total shifts: %d\n", shiftsInsertion);

    // Compare all three
    printf("\n--- All Three Algorithms Compared ---\n");
    copyArray(original, arr, SIZE);
    swapsBubble = bubbleSort(arr, SIZE);
    copyArray(original, arr, SIZE);
    swapsSelection = selectionSort(arr, SIZE);

    printf("\n  Bubble Sort swaps:      %d\n", swapsBubble);
    printf("  Selection Sort swaps: %d\n", swapsSelection);
    printf("  Insertion Sort shifts: %d\n", shiftsInsertion);

    return 0;
}
```

Challenge 2: Code Explanation

Part 1: The Shift vs Swap Distinction

Insertion Sort does **shifts**, not swaps. A shift is a single assignment (`arr[j+1] = arr[j]`), while a swap is three assignments (using a temp). Shifts are roughly **3× cheaper** than swaps.

```
Swap:  temp = a;  a = b;  b = temp;    // 3 assignments
Shift: arr[j+1] = arr[j];              // 1 assignment
```

Even though Insertion Sort's shift count may be higher than Selection Sort's swap count, each operation is cheaper. This is why Insertion Sort is often the fastest simple sort in practice.

Challenge 2: Code Explanation

Part 2: Why Insertion Sort is Adaptive

If the array is **already sorted**, the inner `while` loop never executes -- each `key` is already $\geq$ all elements to its left. Total shifts = 0, total time = $O(n)$.

If the array is **nearly sorted** (few elements out of place), very few shifts are needed -- the inner loop barely runs.

```
Nearly sorted: [1, 2, 4, 3, 5, 6, 8, 7]
Only 3 and 7 need shifting. Shifts: 0+0+0+1+0+0+0+1 = 2.
Nearly  $O(n)$  performance!
```

This makes Insertion Sort the best simple sort for **nearly sorted data** and the reason it is used as the base case in hybrid sorts like Timsort.

Challenge 2: Code Explanation

Part 3: Comparing All Three Algorithms

Property	Bubble Sort	Selection Sort	Insertion Sort
Mechanism	Swap adjacent pairs	Find min, swap to front	Shift elements, insert key
Swaps/Shifts	Many swaps ($O(n^2)$)	Few swaps ($\leq n-1$)	Many shifts but cheap
Best case	$O(n)$ -- sorted input	$O(n^2)$ -- always	$O(n)$ -- sorted input
Stable?	Yes	No	Yes
Adaptive?	Yes (with early stop)	No	Yes
Practical speed	Slowest	Middle	Fastest for small/nearly sorted

Rule of thumb: For small arrays ($n < 20-30$), Insertion Sort wins. For larger arrays, use $O(n \log n)$ algorithms like Merge Sort or Quicksort.

Lab Session Complete!

Summary: What You Built

Challenge 1: Bubble Sort and Selection Sort

- Implemented Bubble Sort with early termination optimization
- Implemented Selection Sort with minimum-finding strategy
- Compared swap counts: Bubble Sort performed more swaps, but Selection Sort always scans the entire unsorted region
- Understood stability: Bubble Sort is stable, Selection Sort is not

Challenge 2: Insertion Sort

- Implemented Insertion Sort using shifts instead of swaps
- Traced the sorted region growing from left to right with each insertion
- Compared all three algorithms: Insertion Sort is the most practical simple sort for small or nearly sorted data

Key Concepts Reinforced

Bubble Sort

- Repeatedly swaps adjacent out-of-order pairs. Largest element "bubbles" to the end each pass.
- **Early termination** when a pass has zero swaps makes it adaptive: $O(n)$ best case.

Selection Sort

- Finds the **minimum** of the unsorted region and swaps it into place.
- Always $O(n^2)$ comparisons. Fewest swaps (at most $n - 1$). Not stable, not adaptive.

Insertion Sort

- Inserts each element into its correct position within the sorted prefix via **shifts**.
- Adaptive ($O(n)$ on sorted data). Stable. Fastest simple sort in practice for small arrays.

When to Use Which

- **Small arrays ($n < 30$):** Insertion Sort.
- **Minimize swaps (write-expensive media):** Selection Sort.
- **Educational purposes:** Bubble Sort (simplest to understand, rarely used in practice).

Complexity Summary

Operation	Best	Average	Worst	Space	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes

How do these compare with advanced sorts?

Algorithm	Average	Worst	Stable?	Notes
Merge Sort	$O(n \log n)$	$O(n \log n)$	Yes	Needs $O(n)$ extra space
Quicksort	$O(n \log n)$	$O(n^2)$	No	Fastest in practice (average)
Timsort	$O(n \log n)$	$O(n \log n)$	Yes	Uses Insertion Sort for small runs

All three simple sorts are $O(n^2)$ -- fine for small inputs, but too slow for large datasets.

Take-Home Challenge (Optional)

Extend your implementations with these exercises:

1. **Count Comparisons:** Modify all three sort functions to also count the total number of comparisons (not just swaps/shifts). On the same input, which algorithm makes the most comparisons? Is it always the same one?
2. **Descending Order:** Modify all three algorithms to sort in **descending** order. What is the minimum change needed? (Hint: flip one comparison operator.)
3. **Nearly Sorted Input:** Test all three algorithms on the array `{1, 2, 3, 5, 4, 6, 7, 9, 8}`. Which algorithm performs the fewest operations? Why? This demonstrates the practical advantage of adaptive algorithms.
4. **Think about it:** Why is Selection Sort preferred when writing to flash memory or other media where **writes are expensive** but reads are cheap? How many writes (swaps) does Selection Sort guarantee compared to Bubble Sort on an array of 1000 elements?

